BRD: Intelligent Personal Productivity Hub Using ML, OpenCV, and RAG

1. Overview

This Business Requirements Document (BRD) describes an "Intelligent Personal Productivity Hub" designed as a final-year computer science project that combines machine learning, OpenCV-based computer vision, a basic Retrieval-Augmented Generation (RAG) pipeline, and a rich software user interface. The system targets students and knowledge workers who struggle to organize and retrieve information from scattered notes, documents, meeting content, and screenshots.[1][2][3][4]

The application will ingest text and visual content from handwritten notes, printed documents, slides, whiteboards, and digital files, then provide a unified interface for search, summarization, task extraction, and context-aware assistance powered by RAG. The goal is to deliver a realistic, deployable product that demonstrates end-to-end engineering skills: data capture, ML/CV processing, backend services, UI design, and integration with a language model.[2][5][6][1]

2. Problem Statement

Students and professionals generate large amounts of unstructured personal data: handwritten notes, PDFs, photos of whiteboards, meeting recordings, and screenshots, which become difficult to search and reuse later. Traditional folder-based storage or cloud drives do not provide semantic understanding, automatic summarization, or task extraction, leading to time wasted on manually locating and rereading materials.[3][4]

Existing RAG-based assistants and productivity tools focus mainly on text documents and lack tight integration with everyday visual inputs like notes captured via camera or real-time screenshots from study or work sessions. There is a need for an integrated personal hub that understands both textual and visual content, organizes it intelligently, and exposes it through a convenient UI with automation workflows.[7][4]

3. Objectives

- Provide a unified platform to capture, store, and process personal knowledge artifacts (notes, documents, meetings, screenshots).
- Use OpenCV-based image processing to clean and structure visual inputs (page detection, enhancement, region detection).
- Apply ML models for document classification, topic modeling, and action-item detection to turn raw text into structured knowledge.
- Implement a basic RAG pipeline over the user’s knowledge base to enable grounded question answering and summarization.[1][2]

- Design a modern, user-friendly UI that supports searching, chatting with the assistant, viewing timelines, and managing tasks.
- Demonstrate scalable architecture suitable for a final-year project with clear modules and workflows.

4. Scope

4.1 In-Scope Features

- Image capture and preprocessing for:
  - Handwritten notes and book pages.
  - Slides and whiteboards captured via screenshots or camera.
- Text extraction via OCR from preprocessed images.
- Ingestion of existing digital text documents (PDF, DOCX, plain text).
- ML-based classification and topic tagging for documents.
- Action-item detection and task creation from notes and meeting content.
- RAG-based question answering over all stored content.
- Rich web or desktop UI for document browsing, chat, timeline, and tasks.

4.2 Out-of-Scope (Optional / Future Work)

- Full audio-based meeting transcription (only optional integration with external APIs).
- Enterprise multi-user deployment and complex role-based access control beyond simple user roles.
- Advanced visual RAG with multimodal embeddings for large-scale image collections.[4][7]

5. Stakeholders and Users

- **Primary user:** Individual student or knowledge worker who wants to manage personal notes and documents.
- **Secondary stakeholders:** Supervisors, mentors, or evaluators who will assess the system's architecture and functionality.
- **System administrator (optional role):** Manages configuration, storage limits, and model selection.

6. High-Level Use Cases

6.1 Capture and Organize Notes

- User takes a photo of handwritten notes with the app.
- System detects the page, corrects perspective, enhances contrast, and runs OCR.
- Text is stored, tagged with topic and document type, and associated with the user.

6.2 Meeting Content Ingestion

- User uploads screenshots of slides or whiteboard or shares a folder containing such images.
- System detects text regions, extracts text via OCR, and segments content by slide or board section.[^8]
- Summaries and key decisions are generated via the language model.

6.3 Ask Questions via RAG

- User asks: "What did we decide in last week's project meeting?" or "Show me all notes about CNN architectures."
- System retrieves relevant chunks from vector storage, constructs a context, and returns a grounded answer.[2][1]

6.4 Task Extraction and Management

- System scans processed notes and meeting content for action items (e.g., "Complete assignment" or "Send email").
- Detected tasks are added to a task list with optional due dates and priorities.

6.5 Knowledge Browsing and Timeline

- User explores a timeline view showing captured notes, meetings, and documents by date.
- User filters content by topic or document type for revision or review.

7. Functional Requirements

7.1 Image Capture and Preprocessing

- The system shall capture images from a webcam or upload from the file system.
- The system shall detect document boundaries using OpenCV (contour detection, edge detection) and perform perspective correction.
- The system shall enhance brightness and contrast and optionally convert to grayscale to improve OCR performance.
- The system shall support batch processing of multiple images.

7.2 OCR and Text Extraction

- The system shall apply OCR to preprocessed images using an open-source engine (e.g., Tesseract or a deep learning-based OCR) to obtain text.
- The system shall handle mixed language text where possible, focusing primarily on English.
- The system shall associate extracted text with the original image and store both.

7.3 Document Ingestion (Digital Files)

- The system shall allow users to upload PDFs, DOCX files, and plain-text documents.
- The system shall extract text from supported formats using appropriate libraries.
- The system shall store extracted text with metadata: title, date, source type (scanned, digital), and tags.

7.4 ML-based Classification and Tagging

- The system shall use an ML model to classify documents into types: lecture note, assignment, project report, bill, etc.
- The system shall perform topic detection or clustering (e.g., using embeddings and unsupervised clustering) to assign topic labels.
- The system shall expose APIs to update or correct tags based on user feedback.

7.5 Action-Item Detection

- The system shall analyze text chunks to identify sentences containing action items (tasks).
- The system shall create task entries with description and source reference (document or meeting).
- The system shall allow users to edit, delete, and mark tasks as complete.

7.6 RAG-based Question Answering

- The system shall chunk extracted text into segments with metadata (source, timestamp, topic).
- The system shall compute embeddings for each segment using a sentence or document embedding model and store them in a vector database.[1][2]
- The system shall implement semantic retrieval to fetch the most relevant segments for a given query.[^1]
- The system shall call a language model with the user query and retrieved segments to generate grounded answers.[3][2]
- The system shall provide response explanations by listing sources used for the answer.

7.7 UI and UX Requirements

- The system shall provide a responsive UI accessible via web browser or desktop framework.
- The UI shall include:
  - A document library view (list or grid) with thumbnails based on preprocessed images.
  - A timeline view showing items by date.
  - A chat interface for interacting with the RAG assistant.
  - A task view with filters (pending, completed, by source).
- The UI shall support search and filtering based on topic, type, and date.
- The UI shall provide settings for users to configure capture sources and model options.

7.8 User Management and Roles

- The system shall support user accounts with authentication.
- The system may optionally support simple roles (User, Admin) for configuration and monitoring, similar to academic project lifecycle platforms that use role-based dashboards.[9][10]

8. Non-Functional Requirements

8.1 Performance

- Image preprocessing and OCR should complete within a reasonable time (e.g., under a few seconds per page on typical hardware).
- Retrieval and response generation should feel interactive, with typical query responses returned within a few seconds depending on model latency.[^2]

8.2 Scalability

- The architecture should allow scaling to thousands of documents using vector databases that support efficient similarity search.[6][1]
- Storage should be organized to allow adding new data sources (e.g., email, code repositories) in future.

8.3 Security

- User data (notes, documents, tasks) should be stored securely with proper authentication and basic authorization.
- Connection to external language model APIs should avoid logging confidential content at third-party endpoints where possible.

8.4 Usability

- The UI should be intuitive with clear navigation, tooltips, and consistent design.
- Onboarding screens or hints should guide first-time users through capturing their first notes and asking their first questions.

9. System Architecture

9.1 High-Level Components

- **Frontend UI:** React, Vue, Angular, or similar framework providing the main dashboard, document views, chat interface, and task manager.[^5]
- **Backend API:** Python-based (FastAPI, Django) or Node.js service layer offering endpoints for image upload, OCR, ML classification, RAG query, and task operations.
- **Processing Services:**
  - Image preprocessing (OpenCV).
  - OCR service.
  - ML service for classification, topic modeling, and action-item detection.
  - RAG pipeline service (embeddings, vector DB, retriever, generator).[2][1]
- **Data Storage:**
  - Relational database (PostgreSQL/MySQL) for metadata, users, tasks.
  - Vector database (FAISS, Chroma, Qdrant) for embeddings and segments.[6][1]
  - File storage for raw images and uploaded documents.

9.2 Integration with Language Models

- The backend will integrate with an LLM API or locally hosted model for summarization, question answering, and action-item extraction.[^2]
- Prompts will be designed to include user queries and selected context segments from the retriever.

10. Detailed Workflows

10.1 Workflow: Capture and Process Notes

1. User opens the capture screen in the UI.
2. User takes a photo via webcam or uploads an image.
3. Backend calls the image preprocessing service:
   - Detects document boundaries using edge and contour detection.
   - Performs perspective correction.
   - Enhances brightness and contrast.

4. Preprocessed image is stored, and OCR service is invoked.
5. OCR extracts text, which is returned to backend and stored with metadata.
6. ML service classifies document type and assigns topic tags.
7. Document appears in the library and relevant entries show up in the timeline UI.

10.2 Workflow: Meeting Content Ingestion

1. User uploads a batch of screenshots or board photos.
2. Backend runs preprocessing (cropping, enhancement) and text-region detection (e.g., using OpenCV methods).
3. OCR extracts text from each region.
4. Text is grouped per meeting session using timestamps or user-provided labels.[^8]
5. Summarization module generates a concise meeting summary and highlights decisions.
6. Action-item detection creates tasks linked to the meeting.
7. All artifacts appear under a meeting entry in the UI.

10.3 Workflow: RAG Question Answering

1. User opens the assistant chat and types a query.
2. Backend sends query to retriever:
    - Embeds query using the same embedding model as documents.[^1]
    - Performs similarity search on the vector database.
    - Returns top-k relevant segments.
3. Backend constructs a prompt including query and retrieved segments.
4. LLM generates a response grounded in those segments.[3][2]
5. Backend sends response to the UI along with source references.
6. UI shows the answer with clickable source links.

10.4 Workflow: Task Management

1. After ingestion, the action-item detection service scans new text segments.
2. Sentences detected as tasks are sent to the backend.
3. Backend creates task entries linked to source documents.
4. User views tasks in the task panel and may set due dates.
5. User marks tasks as complete; status updates in the database.

11. Technology Stack (Indicative)

- **Programming language:** Python for backend and ML/CV; optional JS/TS for frontend.
- **Computer Vision:** OpenCV for preprocessing and region detection.[^8]
- **OCR:** Tesseract or equivalent OCR library.
- **ML/NLP:** Transformer-based models or sentence embedding models; simple classifiers for document type and action-item detection.
- **RAG:** LangChain or custom pipeline orchestrating embeddings, retrieval, and generation.[3][2]
- **Vector DB:** FAISS, Chroma, or Qdrant.[6][1]
- **Relational DB:** PostgreSQL or MySQL.
- **Frontend:** React, Vue, Angular, or similar.[^5]
- **Deployment:** Local machine, Docker-based deployment, or basic cloud VM.

12. Risks and Mitigations

- **OCR quality on poor images:** Risk of inaccurate text from low-light or blurred images. Mitigate through strict preprocessing, user guidance on capture quality, and optional retake prompts.[^4]
- **Model latency:** LLM calls may be slow. Use caching and limit context length; design UI to show loading states.[^2]
- **Data privacy:** Personal notes may be sensitive. Ensure local-only deployment or trusted cloud configurations; avoid unnecessary logging and secure API keys.[^4]
- **Complexity for a single student:** The architecture is rich. Prioritize core modules first (capture, OCR, RAG, basic UI) and treat meeting workflows and advanced automation as stretch goals.[^3]

13. Success Metrics

- Number of documents and notes successfully ingested and searchable.
- Accuracy of document type classification and topic tagging (measured on labeled samples).
- Quality of RAG answers as judged by users (subjective rating or rubric).
- Percentage of correctly detected action items.
- User satisfaction with UI usability and responsiveness.

14. Future Extensions

- Integration of audio-based transcription for full meeting recordings.
- Advanced multimodal RAG combining image embeddings with text, as explored in RAG + vision research.[7][4]
- Collaborative use cases (sharing workspaces with classmates or team members).
- Analytics dashboards showing study time, topic coverage, and productivity trends.

References

1. The Ultimate Guide to Vector DB and RAG Pipeline - Processing long documents with VLMs poses a huge challenge. Document RAG pipeline - Vector DB and RA...
2. Retrieval Augmented Generation – RAG with LLMs - Learn OpenCV - Retrieval Augmented Generation - RAG with LLMs powered by Langchain and the Hugging Face Transformer...
3. Top 25+ RAG Project Ideas in 2026 - Agentic AI - Explore RAG project Ideas with real-world ideas like chatbots, document assistants, and AI search ap...
4. RAG and Computer Vision Applications in AI - Ultralytics - Learn how combining retrieval-augmented generation (RAG) with computer vision is helping AI systems ...
5. Best AI Automation Tools for Final Year Projects – 2026 Students - Discover top AI automation tools for final year projects. Automate workflows, testing, and AI applic...
6. Awesome list of Retrieval-Augmented Generation (RAG) ... - 😎 Awesome list of Retrieval-Augmented Generation (RAG) applications in Generative AI. - Danielskry/A...
7. RAG for Vision: Building Multimodal Computer ... - Medium - We explore what Visual RAG is, why it matters, and how it’s supercharging a Computer Vision Pipeline...
8. 20+ Real-Time AI & Computer Vision Projects — Powered by Mediapipe & OpenCV! - Explore the future of gesture control, pose detection, and intelligent automation with our complete ...
9. IntelliFYP: AI Powered Final Year Projects Lifecycle Automation & Archival System - Namal CDC - IntelliFYP is an AI-powered, role-based full-stack web platform designed to automate and manage the ...
10. #finalyearproject #ai #softwareengineering #dotnet #angular ... - Excited to share the demo video of our Final Year Project, IntelliFYP — an AI-powered platform built...